

AI Damage Assessment Feature

Module 1, Feature 1 - Implementation & Architecture Report

Date: August 2026 | Platform: Decorate3D C2C Furniture Marketplace

1. Executive Summary

The AI Damage Assessment feature (Module 1, Feature 1) introduces automated furniture inspection to the Decorate3D marketplace. By integrating the state-of-the-art Google Gemini Vision API (gemini-3.5-flash model), sellers can upload photographs of their used furniture and obtain an instantaneous, objective, and visual damage report. The system automatically detects scratches, tears, dents, stains, structural cracks, and wear, overlaying red boundary highlights on the image, assigning a condition grade, and recommending fair valuations based on the structural and aesthetic integrity of the item.

2. Architecture & Data Flow

The implementation utilizes a modern, non-blocking asynchronous architecture. Below is a summary of the core components created and updated in the project codebase:

*** Backend API Endpoint (damage.controller.js & damage.routes.js):**

- Exposes POST /api/modules/m1/ai-damage-assessor/assess route.
- Accepts standard multipart form-data image uploads or JSON payloads containing base64 images.
- Reuses the global, secure config/multer.js configuration to write assets to static public directories.
 - Executes vision inspection by passing base64 buffers directly to Gemini Generative API with forced schema constraint.

*** Frontend Widget (DamageAssessorWidget.jsx):**

- React interface that renders drag-and-drop file inputs and displays local image previews.
- Integrates a scanning horizontal laser line animation for feedback during the asynchronous API call.
- Reads percentage-based coordinates and dynamically draws overlay boxes on absolute-positioned HTML elements.
- Features mouse-hover sync highlighting individual defects and showing detailed popover tooltips.

3. Gemini Vision API & Schema Enforcement

To guarantee predictability and prevent parsing exceptions in the Express backend, the model enforces structured JSON schema validation. By using Gemini's `responseMimeType: 'application/json'` and passing a JSON schema configuration, the API strictly returns objects matching our formatting needs.

The enforced schema definition configuration passed in API request is as follows:

```
{
  "conditionGrade": "FAIR | GOOD | EXCELLENT",
  "confidenceScore": 0-100,
  "summary": "Brief sentence summarizing damage",
  "damages": [
    {
      "type": "scratch | dent | tear | stain | structural_crack | wear",
      "severity": "minor | moderate | severe",
      "description": "Defect detail details",
      "boundingBox": [ymin, xmin, ymax, xmax] // 0-1000 scale
    }
  ]
}
```

4. Coordinate Mapping & Absolute Overlays

Google Gemini API coordinates are normalized on a scale of 0 to 1000, where [0,0] represents the top-left and [1000, 1000] the bottom-right corners. For seamless responsive rendering on front-end components, the controller converts these to percentages before sending the response.

Normalizing logic applied in the backend `damage.controller.js`:

- * top (%) = $ymin / 10$
- * left (%) = $xmin / 10$
- * bottom (%) = $ymax / 10$
- * right (%) = $xmax / 10$

Responsive styling logic in the `React DamageAssessorWidget.jsx`:

- * width (%) = $right - left$
- * height (%) = $bottom - top$
- * position = absolute (relative to parent container)

5. Regression Pricing Engine Integration

This feature integrates directly into Module 2's Regression Pricing Engine. When the AI completes a damage assessment scan, it calculates the condition grade (EXCELLENT, GOOD, FAIR).

This grade is immediately applied to the `SellerListingModal` state. The modal debounces the inputs and triggers a fetch to the pricing controller endpoint at `/api/modules/m2/price-recommendation`. The pricing calculator uses age decay and condition coefficients (EXCELLENT: 0.75, GOOD: 0.55, FAIR: 0.35) to suggest listing price ranges automatically. This connects the vision API outcome to marketplace transactional metrics, guaranteeing fair pricing.

6. API Reference

POST /api/modules/m1/ai-damage-assessor/assess

Headers: Content-Type: multipart/form-data OR application/json

Success Response Example (JSON):

```
{
  "success": true,
  "feature": "AI Damage Assessment",
  "conditionGrade": "FAIR",
  "confidenceScore": 92,
  "summary": "The leather armchair has a visible tear on the side seat cushioning.",
  "imageUrl": "/uploads/images/sample-1723456789.jpg",
  "damages": [
    {
      "type": "tear",
      "severity": "moderate",
      "description": "A 3-inch tear exposing inner cushioning on outer right arm.",
      "boundingBox": [420, 110, 560, 310],
      "boundingBoxPercent": {
        "top": 42.0,
        "left": 11.0,
        "bottom": 56.0,
        "right": 31.0
      }
    }
  ]
}
```

7. Local Setup & Verification

To run the damage assessment feature in a local development environment, follow these steps:

1. Add GEMINI_API_KEY=your_actual_key to the local .env configuration file.
2. Make sure the backend server and Vite dev client are running (npm run server / npm run dev).
3. Open the browser to the seller hub /seller, select 'List New Item', and click the upload zone under 'AI Vision Damage Scanner' to upload furniture images and run the analysis.

--- End of Document ---